areas of your app too. If your app doesn't verify that the URL is safe before redirecting the user, it could redirect users to malicious sites.

The simple solution to this attack is to always validate that the `returnUrl` is a local URL that belongs to your app before redirecting users to it. The default Identity UI does this already, so you shouldn't have to worry about the login page if you're using Identity, as described in chapter 23.

If you have redirects in other parts of your app, ASP.NET Core provides a couple of helper methods for staying safe, the most useful of which is `Url.IsLocalUrl()`. Listing 29.5 shows how you could verify that a provided return URL is safe and, if not, redirect to the app's home page.

You can also use the `LocalRedirect()` helper method on the `ControllerBase` and Razor Page `PageModel` classes, which throw an exception if the provided URL isn't local.

Listing 29.5 Detecting open redirect attacks by checking for local return URLs

```
[HttpPost]
public async Task<IActionResult> Login(
    LoginViewModel model, string returnUrl = null)   ❶
{
    // Verify password, and sign user in

    if (Url.IsLocalUrl(returnUrl))                   ❷
    {
        return Redirect(returnUrl);                  ❸
    }
    else
    {
        return RedirectToPage("Index");              ❹
    }
}
```